

# Makefile

Laboratórios de Informática III

Guião #2 - 1

Departamento de Informática  
Universidade do Minho

2025/26

## Conteúdo

|          |                                                                |          |
|----------|----------------------------------------------------------------|----------|
| <b>1</b> | <b>Estrutura básica</b>                                        | <b>2</b> |
| <b>2</b> | <b>Compilação e linkagem</b>                                   | <b>3</b> |
| <b>3</b> | <b>Regras implícitas e variáveis especiais</b>                 | <b>3</b> |
| <b>4</b> | <b>Objetivos PHONY</b>                                         | <b>4</b> |
| <b>5</b> | <b>Extra: opções de caminhos e bibliotecas para compilação</b> | <b>5</b> |
| <b>6</b> | <b>Exemplo completo</b>                                        | <b>6</b> |

# 1 Estrutura básica

Um Makefile é um ficheiro utilizado para facilitar a compilação de programas C de maior dimensão. Ele permite não só especificar os vários passos de compilação de uma forma legível, mas também garantir que ao recompilar um programa, apenas os ficheiros que sofreram alterações são novamente compilados, tornando o processo mais eficiente.

É executado através do comando `make`, e deve ter o nome `Makefile` para que possa ser corretamente identificado pelo comando <sup>1</sup>. Este ficheiro é composto por um conjunto de regras. De um modo geral, cada regra especifica como devem ser compilados os ficheiros de código-fonte e como ligar ("*linkar*") esses ficheiros objeto de forma a serem produzidos um ou mais executáveis<sup>2</sup>. Assim sendo, do processamento bem sucedido de um Makefile resulta normalmente a produção de um ou mais programas executáveis (bem como de um conjunto de ficheiros intermédios).

```
1 objetivo: pré-requisitos
2     receita
```

Cada regra é composta por um objetivo, um ou mais pré-requisitos, e uma receita. Os **pré-requisitos** são um conjunto de ficheiros (`.o` ou `.c`) e/ou objetivos de outras receitas necessários à execução de uma regra.

O **objetivo** é normalmente constituído por um ficheiro objeto ou executável que se obtém da execução da receita.

A **receita** é um conjunto de instruções/comandos do interpretador de comandos ("*shell*") que, quando aplicados aos ficheiros descritos pelos pré-requisitos, resultam na produção do objetivo. Os pré-requisitos e objetivo devem estar escritos na mesma linha. A receita pode ocupar várias linhas, uma por cada comando a executar. Cada linha de uma receita tem necessariamente de começar por um **tab (muito importante!)**.

```
1 # O executável main tem como pré-requisito a existência do ficheiro main.c.
2 # A receita compila o ficheiro main.c, tendo como resultado o executável denominado main.
3 main: main.c
4     gcc main.c -o main
```

Diferentes regras podem ser encadeadas através dos pré-requisitos. Para executar uma regra, é necessário que todos os seus pré-requisitos sejam preenchidos. Para isso, é possível que seja necessário executar outras regras de forma a atingir esses pré-requisitos.

```
1 # Esta regra precisa do ficheiro main.o, pelo que irá executar a regra abaixo
2 main: main.o
3     gcc main.o -o main
4
5 # Regra que produz (tem como objetivo) o ficheiro main.o
6 main.o: main.c
7     gcc -c main.c -o main.o
```

Ao chamar o comando `make`, sem qualquer argumento, a primeira regra no ficheiro é executada. Por convenção, à primeira regra é dado o nome de *all*. Quando o comando `make` é executado com um argumento, a regra cujo objetivo coincide com esse argumento é executada.

```
1 all: main
2
3 # Esta regra necessita do ficheiro main.o, pelo que irá executar a regra abaixo
4 main: main.o
5     gcc main.o -o main
6
7 # Regra que produz (tem como objetivo) o ficheiro main.o
8 main.o: main.c
9     gcc -c main.c -o main.o
10
11 clean:
12     rm -f main main.o
```

<sup>1</sup>É possível especificar um nome de ficheiro diferente usando a opção `-f <filename>` como argumento da linha de comando.

<sup>2</sup>Estas regras podem também descrever outros "produtos" do projeto, tais como, por exemplo, documentação, relatórios, etc.

No excerto acima, o comando `make` produz o executável com o nome `main`. Já o comando `make clean` irá executar a última regra que remove os ficheiros `main` e `main.o`.

## 2 Compilação e linkagem

Por motivos de eficiência e legibilidade, deve fazer-se a separação entre passos de compilação de ficheiros objeto, e o processo de linkagem. Para isso, faz-se uso de três **variáveis**, `CC`, `CFLAGS` e `LDFLAGS` para, respetivamente: especificar o compilador a usar; especificar as *flags* a usar para compilação; e especificar as *flags* a usar para linkagem. As variáveis são definidas no início do ficheiro, antes da primeira regra, e podem ser usadas nas regras utilizando o formato `$(NOME_VARIAVEL)`.

```

1  # Variável CC define o compilador a usar
2  CC = gcc
3  # Variável LDFLAGS define flags de linkagem
4  LDFLAGS =
5  # Variável CFLAGS define flags de compilação. Notar a opção -c para gerar ficheiros .o
6  CFLAGS = -Wall -c
7
8  # O executável main tem como pré-requisito a existência dos ficheiros main.o e deque.o.
9  # A receita faz a linkagem dos ficheiros objeto, utilizando as variáveis de flags
10 # de linkagem LDFLAGS, tendo como resultado o executável denominado main.
11 # Tem como pré-requisitos os ficheiros main.o e deque.o, cujas regras serão executadas
12 # caso esses ficheiros não existam ou os seus ficheiros fonte tenham sido alterados
13
14 all: main
15
16 # Passo de linkagem
17 main: main.o deque.o
18     $(CC) main.o deque.o $(LDFLAGS) -o main
19
20 # Passo de compilação
21 main.o: main.c
22     $(CC) $(CFLAGS) main.c
23
24 # Passo de compilação
25 deque.o: deque.c
26     $(CC) $(CFLAGS) deque.c -o deque.o
27
28 # Comando de limpeza
29 clean:
30     rm -f deque.o main.o main

```

## 3 Regras implícitas e variáveis especiais

Existe um conjunto de regras (e receitas) implícitas que são aplicadas por omissão<sup>3</sup> no processamento de um Makefile nos casos em que este não as especifica de forma explícita. Por exemplo, no caso de regras que têm como objetivo a compilação de código-fonte e geração dos respetivos ficheiros objeto (.o) a seguinte lógica é usada.

```

1  main.o: main.c
2
3  # Por omissão executa a mesma receita que:
4  main.o: main.c
5      $(CC) $(CPPFLAGS) $(CFLAGS) -c main.c -o main.o

```

Assim, podemos escrever:

<sup>3</sup>A lista completa de regras e receitas implícitas e explícitas aplicadas no processamento de um Makefile pode ser obtida através da inclusão da opção `-p` nos argumentos do comando `make`.

```

1 CC = gcc
2 LDFLAGS =
3 CFLAGS = -Wall -c
4
5 all: main
6
7 # Passo de linkagem
8 main: main.o deque.o
9     gcc main.o deque.o $(LDFLAGS) -o main
10
11 # Passo de compilação
12 main.o: main.c
13
14 # Passo de compilação
15 deque.o: deque.c
16
17 # Comando de limpeza
18 clean:
19     rm deque.o main.o main

```

Existe também um conjunto de variáveis especiais que podem, opcionalmente, ser usadas para tornar o código mais uniforme e sucinto. Por exemplo:

- `$@` - objetivo da regra.
- `$^` - todos os pré-requisitos separados por espaço.
- `$<` - primeiro pré-requisito.

Assim, o passo de linkagem do exemplo anterior pode ser escrito da seguinte forma:

```

1 # Passo de linkagem
2 main: main.o deque.o
3     gcc $^ $(LDFLAGS) -o $@

```

## 4 Objetivos PHONY

Os objetivos PHONY são uma funcionalidade importante de um Makefile que permite definir regras que não correspondem a ficheiros reais no sistema. Por omissão, o make assume que cada regra produz um ficheiro com o mesmo nome do objetivo. No entanto, existem casos em que queremos definir regras que executam comandos sem criar ficheiros correspondentes.

O problema surge quando existe um ficheiro com o mesmo nome do objetivo. Por exemplo, se tivermos uma regra `clean` e existir um ficheiro chamado `clean` na diretoria, o make pode não executar a regra porque considera que o objetivo (nem sequer é um objetivo real) já está atualizado.

Para evitar este problema, usamos a diretiva `.PHONY:`



```

1 .PHONY: all clean
2
3 CC = gcc
4 CFLAGS = -Wall -c
5
6 all: main
7
8 main: main.o
9     $(CC) main.o -o main
10
11 main.o: main.c
12     $(CC) $(CFLAGS) main.c

```

```

13
14 clean:
15     rm -f main main.o

```

## 5 Extra: opções de caminhos e bibliotecas para compilação

É importante salientar que, para projetos de maior dimensão, as opções `-I` `-L` `-l` do gcc são de especial importância para garantir que os ficheiros são corretamente compilados e linkados.

Na fase de compilação, é necessário que os ficheiros consigam encontrar corretamente os ficheiros `.h` dos quais fazem *include*. Para isso, deve ser usada a opção `-I`.

- `-I`dir adiciona a diretoria *dir* à lista de diretorias a pesquisar no momento de compilação de um ficheiro. As diretorias especificadas com esta opção têm prioridade em relação às pré-definidas pelo sistema.

De notar que **não deve existir nenhum espaço entre a opção e o caminho da diretoria especificada**. Por defeito, o compilador procura por ficheiros *header* nos caminhos pré-definidos pelo sistema, e na mesma pasta em que o comando gcc está a ser executado. Para utilizar ficheiros *header* que não estejam em nenhum destes caminhos (e.g., ficheiros *header* definidos localmente dentro de sub-pastas) é necessário utilizar esta opção.

No momento de linkagem, é necessário indicar ao compilador onde se encontram as bibliotecas que efetivamente implementam as funções declaradas nos ficheiros *header*, e que bibliotecas devem ser consideradas. Para isso, são usadas as opções `-L` `-l`.

- `-L`dir adiciona a diretoria *dir* à lista de diretorias a pesquisar para `-l`.
- `-l`library incluir a biblioteca *library* no processo de linkagem.

O gcc considera um conjunto de caminhos de bibliotecas e de bibliotecas pré-definido, durante o processo de linkagem. Para utilizar bibliotecas que não as pré-definidas, e.g., glib, é necessário utilizar as opções acima descritas. É importante notar que **a ordem com que é feita a linkagem pode afetar o processo de compilação**. As bibliotecas e ficheiros objetos dependentes devem ficar à esquerda das bibliotecas das quais dependem na sequência de linkagem. Veja-se o exemplo abaixo:

```

# A biblioteca glib é usada pelo módulo "glib_hash_table_test",
# logo deve surgir após esse módulo no processo de linkagem

```



```

gcc main.o glib_hash_table_test.o -lglib2.0 -o hash_table_test

```

Vejamos um exemplo que requer o uso destas opções. Considere a seguinte estrutura de ficheiros:

```

programa
-| Makefile
-| src
    --| main.c
    --| deque.c
-| includes
    --| deque.h

```



Imaginemos ainda que a estrutura deque faz uso da glib.

O ficheiro Makefile tomaria a seguinte forma:

```

1 CC = gcc
2 # Diretoria "includes" e diretoria para headers da glib adicionados às diretorias onde
3 # o compilador deve procurar por headers
4 CFLAGS = -Wall -c -Iincludes -Ipath/to/glib/include
5 # Diretoria da biblioteca da glib e biblioteca glib adicionadas às diretorias onde
6 # o compilador deve procurar por bibliotecas
7 LDFLAGS = -Lpath/to/glib/library -lglib-2.0
8
9 all: main
10

```

```

11 # Passo de linkagem
12 main: src/main.o src/deque.o
13     gcc src/main.o src/deque.o $(LDFLAGS) -o main
14
15 # Passo de compilação
16 src/main.o: src/main.c
17
18 # Passo de compilação
19 src/deque.o: src/deque.c
20
21 # Comando de limpeza
22 clean:
23     rm src/deque.o src/main.o main

```

A ferramenta `pkg-config` é útil na gestão das opções acima mencionadas. Uma vez que os caminhos de bibliotecas e de ficheiros *header* de bibliotecas instaladas variam de sistema para sistema, esta ferramenta permite que os caminhos sejam corretamente definidos, desde que a ferramenta esteja instalada e corretamente configurada no sistema (disponível por omissão em Ubuntu). As opções `--cflags` e `--libs` devem ser usadas para obter as configurações corretas para os processos de compilação e linkagem, respetivamente. Para além disso, deve ser especificada a biblioteca que se quer configurar.

Assim, para o caso da biblioteca `glib` do exemplo anterior, as variáveis poderiam ser definidas da seguinte forma:

```

1 CC = gcc
2 # De notar que o comando pkg-config deve estar envolto pelos acentos
3 CFLAGS = -Wall -c -Iincludes `pkg-config --cflags glib-2.0`
4 # Diretoria da biblioteca do glib e biblioteca glib adicionadas às diretorias onde
5 # o compilador deve procurar por bibliotecas
6 LDFLAGS = `pkg-config --libs glib-2.0`

```

## 6 Exemplo completo

```

1 # Compilador usado
2 CC = gcc
3
4 # Flags a usar para compilação
5 # -Wall: mostrar todos os warnings
6 # -Iinclude: diretoria para os ficheiros header locais
7 # -`pkg-config --cflags glib-2.0`: diretorias dos ficheiros header da glib (opcional, só
8 ↪ necessário se usar glib)
9 CFLAGS = -Wall -Iinclude `pkg-config --cflags glib-2.0`
10
11 # Flags a usar para linkagem
12 # -`pkg-config --libs glib-2.0`: biblioteca glib (opcional, só necessário se usar glib)
13 # -lm: biblioteca matemática (opcional, só necessário se usar funções matemáticas)
14 LDFLAGS = `pkg-config --libs glib-2.0` -lm
15
16 # Executável chamado 'program'.
17 #
18 # Para o gerar, linkamos todos os ficheiros .o e as bibliotecas glib e math.
19 # Usamos $(NAME) para usar a variável NAME.
20 # $@ é equivalente ao objetivo. Neste caso, $@ avalia para: program
21 # $^ significa todos os pré-requisitos: Neste caso, avalia para: src/main.o src/file1.o src/file2.o
22 # Neste exemplo, temos a seguinte regra:
23 # gcc `pkg-config --libs glib-2.0` -lm -Wall -Iinclude `pkg-config --cflags glib-2.0` src/main.o
24 ↪ src/file1.o src/file2.o -o program
25 #
26 # Como esta é a primeira regra, será a que é chamada ao executar 'make' sem argumentos.
27 # Ou seja, 'make' é equivalente a 'make program'.
28 #

```

```

27 # Para executar esta regra, o make irá primeiro precisar de executar as regras 'src/main.o',
   ↪ 'src/file1.o',
28 # e 'src/file2.o', como especificado na lista de pré-requisitos (à direita)
29 program: src/main.o src/file1.o src/file2.o
30     $(CC) $(CFLAGS) $~ $(LDFLAGS) -o $@
31
32 # Aqui começam as regras de compilação dos ficheiros fonte.
33 # Uma regra vazia é equivalente a:
34 # $(CC) $(CPPFLAGS) $(CFLAGS) -c <left param> <right param>
35 # Sendo assim, definimos o objetivo (left) como o nome do ficheiro objeto a ser gerado
36 # e o nome do ficheiro fonte na lista de pré-requisitos (right).
37
38 # Esta regra específica é equivalente a:
39 # gcc -Wall -Iinclude `pkg-config --cflags glib-2.0` -c -o src/main.o src/main.c
40 src/main.o: src/main.c
41
42 # Equivalente à regra acima
43 # src/main.o: src/main.c
44 #     $(CC) $(CFLAGS) -c -o $@ $<
45
46 src/file1.o: src/file1.c
47
48 src/file2.o: src/file2.c
49
50 # Objetivo padrão que compila tudo
51 all: program
52
53 # A regra de limpeza vai remover o executável ('program') e os ficheiros objeto (*.o).
54 # Para a executar, devemos usar 'make clean'.
55 clean:
56     rm -f program src/*.o

```